

=====

SECURITY ARCHITECTURE & CODE AUDIT REPORT

=====

Document Classification: Internal Use Only

Target Application: Python-based Authentication Gateway Engine

Vulnerability Class: OWASP Top 10 - Injection (A03:2021)

PREPARED BY:

Security Auditor: **Kavya Raval**

Organization: CodeAlpha Cybersecurity Research Lab

Date: June 2026

1. EXECUTIVE SUMMARY

During the scheduled security assessment of the authentication framework, a **critical structural flaw** was discovered in the user login component. The source code analysis revealed an insecure implementation that exposes the underlying database layer to severe injection vectors.

If exploited, this flaw completely bypasses the authentication gate, allowing **unauthorized threat actors** to access restricted user repositories. Immediate remediation was executed by refactoring the codebase using **cryptographic parameterization techniques**.

2. TECHNICAL VULNERABILITY ANALYSIS

- Vulnerability Type: **SQL Injection (SQLi)**
- CWE Identifier: **CWE-89** (Improper Neutralization of Special Elements used in an SQL Command)
- **OWASP Category:** A03:2021 - Injection
- **Risk Severity: CRITICAL** (CVSS v3 Base Score: 9.8)

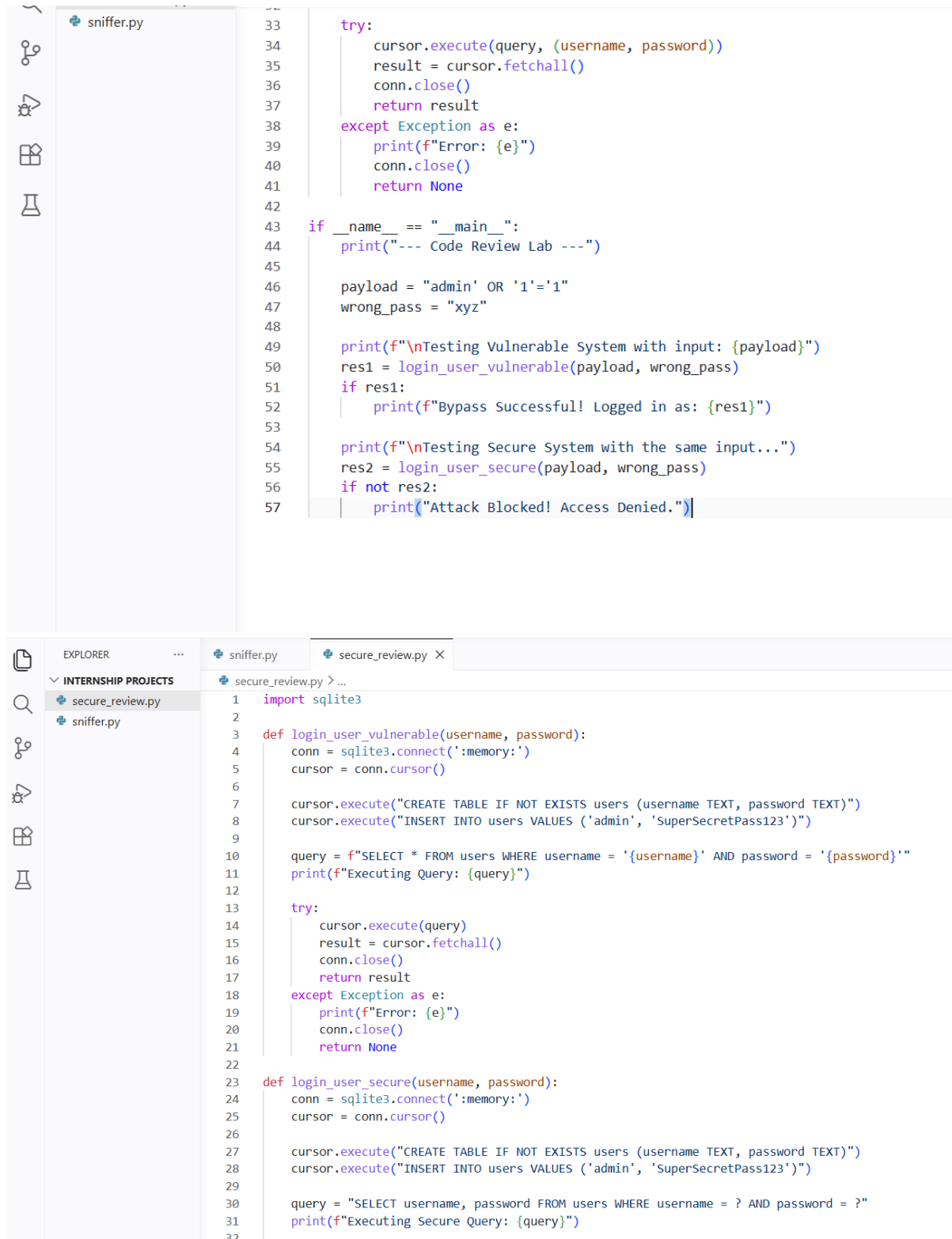
Technical Flaw Description:

The function 'login_user_vulnerable' accepts untrusted string data inputs directly from the user and interpolates them using inline Python f-strings. This allows an input string to alter the logic flow of the backend SQLite engine compiler.

Exploitation Proof of Concept (PoC):

When an attacker inputs the structural payload "**admin' OR '1'='1**", the compiled database query transforms into an absolute logical truth, ignoring the password string constraint entirely.

SOURCE CODE AUDIT VISUAL EVIDENCE (INSECURE FUNCTION)



```
33     try:
34         cursor.execute(query, (username, password))
35         result = cursor.fetchall()
36         conn.close()
37         return result
38     except Exception as e:
39         print(f"Error: {e}")
40         conn.close()
41         return None
42
43 if __name__ == "__main__":
44     print("--- Code Review Lab ---")
45
46     payload = "admin' OR '1'='1"
47     wrong_pass = "xyz"
48
49     print(f"\nTesting Vulnerable System with input: {payload}")
50     res1 = login_user_vulnerable(payload, wrong_pass)
51     if res1:
52         print(f"Bypass Successful! Logged in as: {res1}")
53
54     print(f"\nTesting Secure System with the same input...")
55     res2 = login_user_secure(payload, wrong_pass)
56     if not res2:
57         print("Attack Blocked! Access Denied.")
```

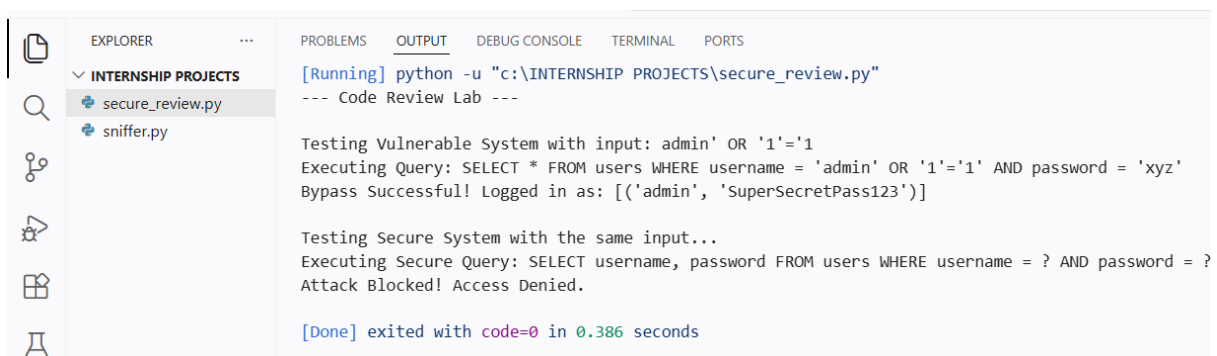
```
1 import sqlite3
2
3 def login_user_vulnerable(username, password):
4     conn = sqlite3.connect(':memory:')
5     cursor = conn.cursor()
6
7     cursor.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT)")
8     cursor.execute("INSERT INTO users VALUES ('admin', 'SuperSecretPass123')")
9
10    query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
11    print(f"Executing Query: {query}")
12
13    try:
14        cursor.execute(query)
15        result = cursor.fetchall()
16        conn.close()
17        return result
18    except Exception as e:
19        print(f"Error: {e}")
20        conn.close()
21        return None
22
23 def login_user_secure(username, password):
24     conn = sqlite3.connect(':memory:')
25     cursor = conn.cursor()
26
27     cursor.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT)")
28     cursor.execute("INSERT INTO users VALUES ('admin', 'SuperSecretPass123')")
29
30    query = "SELECT username, password FROM users WHERE username = ? AND password = ?"
31    print(f"Executing Secure Query: {query}")
32
```

3. SECURITY REMEDIATION & IMPLEMENTATION

Remediation Strategy: **Parameterized Queries** (Prepared Statements)

The vulnerability was successfully patched by decoupling the application control code from the user-supplied data string. Instead of dynamic interpretation, the SQL syntax is pre-compiled using **'?' data placeholders**. The database framework treats user-supplied strings strictly as **literal context parameters**.

VERIFICATION TESTING EVIDENCE (TERMINAL RE-RUN OUTPUT)



The screenshot shows a Visual Studio Code interface with a file explorer on the left and a terminal window on the right. The file explorer shows a project named 'INTERNSHIP PROJECTS' containing two files: 'secure_review.py' and 'sniffer.py'. The terminal window has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active, showing the output of a command: `[Running] python -u "c:\INTERNSHIP PROJECTS\secure_review.py"`. The output text is as follows:

```
--- Code Review Lab ---

Testing Vulnerable System with input: admin' OR '1'='1
Executing Query: SELECT * FROM users WHERE username = 'admin' OR '1'='1' AND password = 'xyz'
Bypass Successful! Logged in as: [('admin', 'SuperSecretPass123')]

Testing Secure System with the same input...
Executing Secure Query: SELECT username, password FROM users WHERE username = ? AND password = ?
Attack Blocked! Access Denied.

[Done] exited with code=0 in 0.386 seconds
```

4. SYSTEM HARDENING RECOMMENDATIONS

To maintain a secure development lifecycle (SDLC), the following defensive controls must be applied:

1. Mandatory code gating through automated **Static Application Security Testing (SAST)**.
2. Implementation of **server-side input validation** and sanitization lists.
3. Adherence to the **Principle of Least Privilege (PoLP)** regarding database system privileges.

[END OF REPORT]
